Laboratorio de Computación
Salas A y B

Profesor(a): Carlos Chaves Mercado

Asignatura: Fundamentos de programación

Grupo: 8

No de Práctica(s): Práctica de estudio 09: Arreglos unidimensionales

Integrante(s): Nolasco Ramírez Sofía

Macay Martínez David

Cruz Gómez Lizette Jacqueline

No. de lista o brigada: Mesa 6

Semestre: 2026-2

Fecha de entrega: 24 de abril de 2026

Observaciones:

CALIFICACIÓN: _____

Objetivo:

El alumno utilizará arreglos de una dimensión en la elaboración de programas que resuelvan problemas que requieran agrupar datos del mismo tipo, alineados en un vector o lista.

Introducción:

Dentro del campo de la ingeniería y el desarrollo de software, la gestión eficiente para cantidades de información grandes es sumamente importante. El uso de variables simples permite almacenar datos de forma aislada, por otro lado, existen casos en los que un problema requiere que se manipule una colección de elementos que compartan cosas en común. Por lo tanto, es bajamente recomendado el uso de variables individuales al ser poco escalables e ineficientes.

Los arreglos unidimensionales, conocidos también como vectores o listas, son estructuras de datos estáticas y homogéneas que permiten almacenar un conjunto de elementos del mismo tipo bajo un nombre común. Se puede visualizar como una sucesión finita de elementos dispuestos en una sola fila o columna, donde cada componente es identificado mediante un índice que representa su posición relativa dentro de la memoria asignada por el compilador.

Para el lenguaje de programación C, el uso e implementación de arreglos para la resolución de distintos problemas optimiza el control de flujo, debido a que permite el uso de estructuras de repetición para recorrer, buscar o modificar datos de manera masiva. A lo largo de esta práctica se pretende dominar la declaración, inicialización y manipulación de estos vectores, comprendiendo la importancia del inicio en la posición 0 en C y la gestión de la memoria para la resolución de problemas lógicos y numéricos complejos.

Desarrollo:

Programa 1a.c
Programa que genera un arreglo unidimensional de 5 elementos y que para poder acceder, recorrer y mostrar cada elemento del arreglo se usa la variable indice desde 0 hasta 4 haciendo uso de un ciclo while.

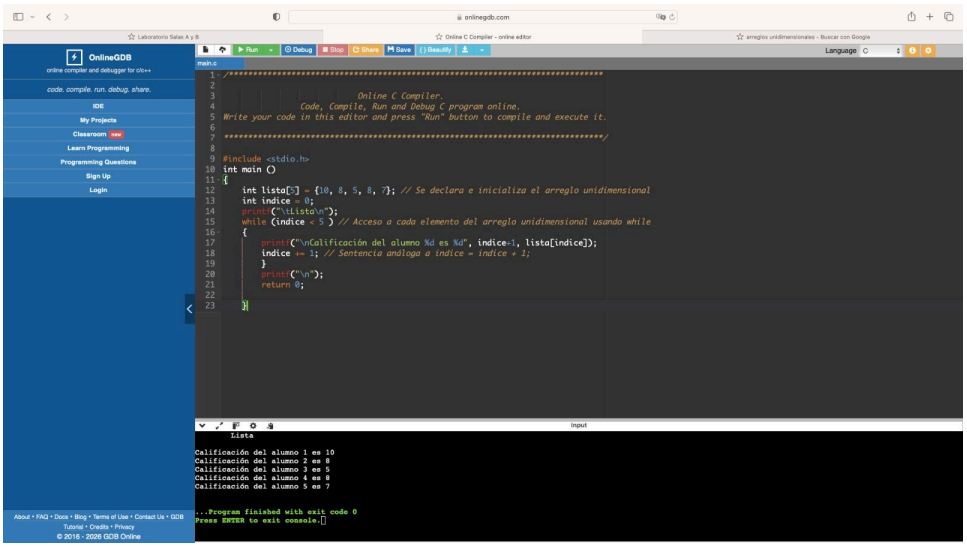


Figura 1: Arreglo unidimensional empleando la estructura de control while.

Programa 1b.c

Se genera un arreglo unidimensional de 5 elementos y que para poder acceder, recorrer y mostrar cada elemento del arreglo se usa la variable indice desde 0 hasta 4 haciendo uso de un ciclo do-while.

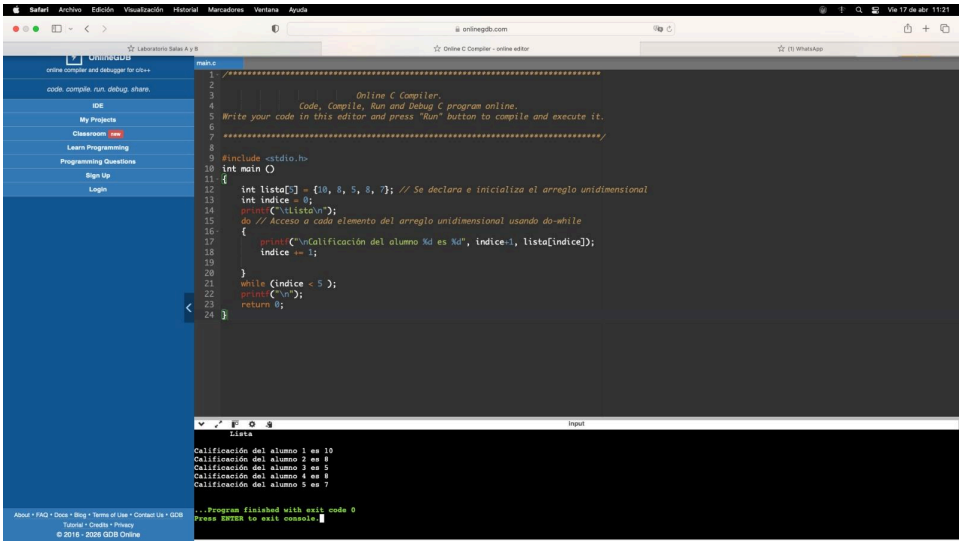


Figura 2: Arreglo unidimensional empleando la estructura de control do-while.

Programa 1c.c

Genera un arreglo unidimensional de 5 elementos y que para poder acceder, recorrer y mostrar cada elemento del arreglo usa la variable índice desde 0 hasta 4 haciendo uso de un ciclo for.

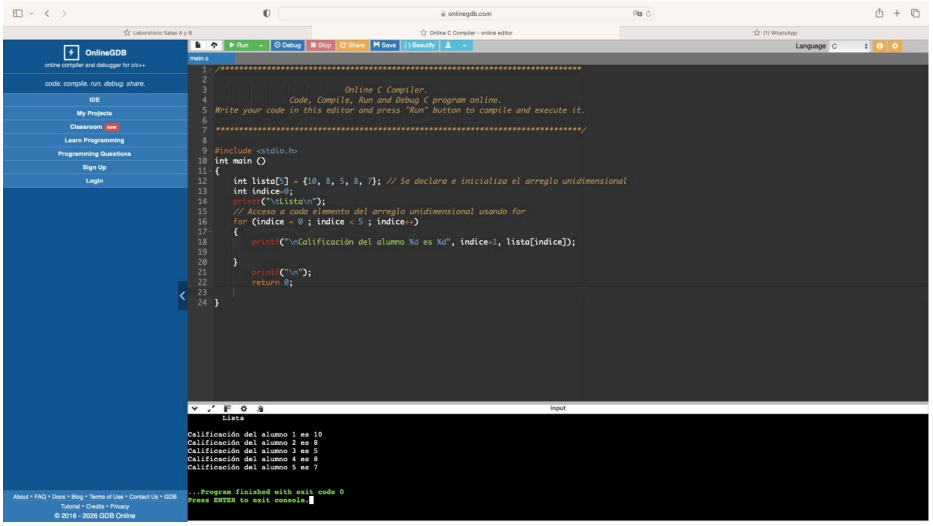
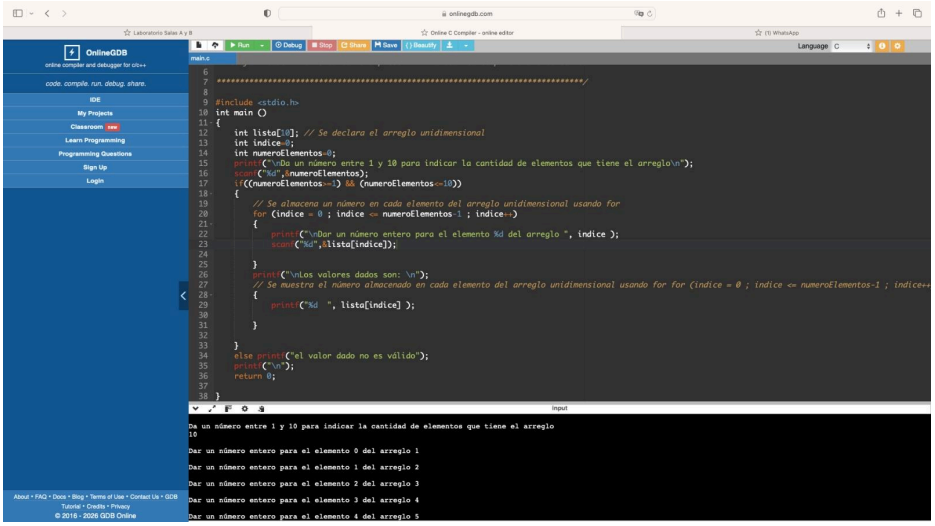
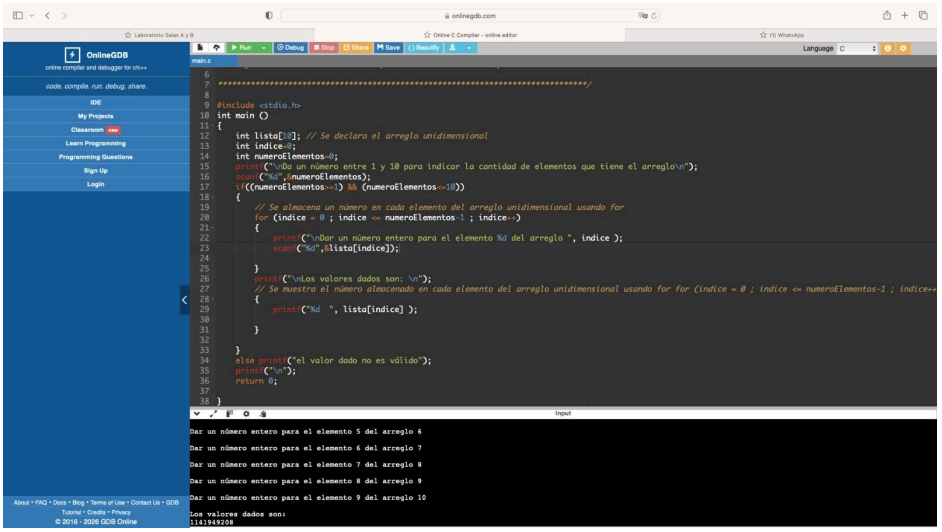


Figura 3: Arreglo unidimensional empleando la estructura de control for.

Programa 2.c

El siguiente programa genera un arreglo unidimensional de máximo 10 elementos. Para poder leer y almacenar datos en cada elemento y posteriormente mostrar el contenido de estos elementos se hace uso de un ciclo for respectivamente.





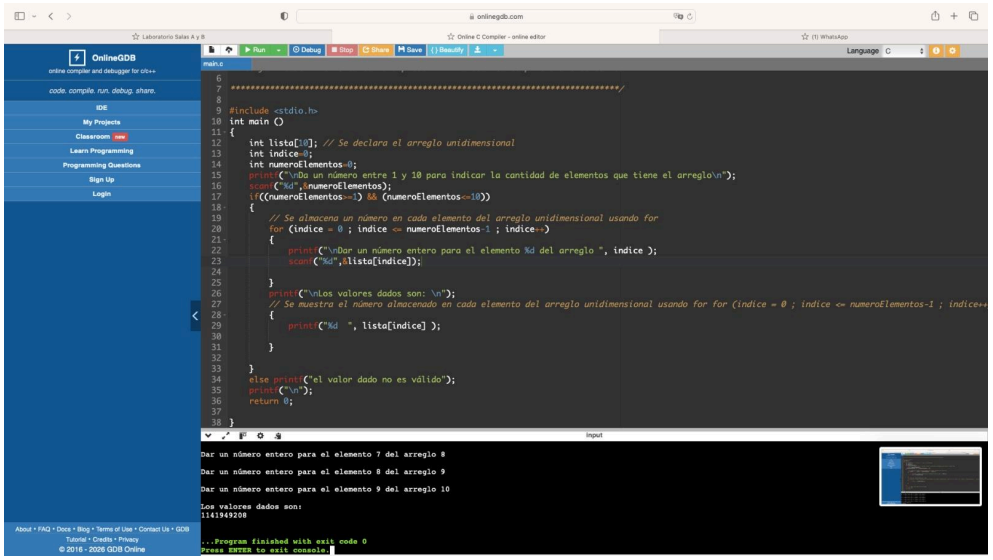


Figura 4: Implementación de un arreglo unidimensional con lectura dinámica de datos y almacenamiento mediante ciclos.

Programa 3.c

Se aprecia la declaración de un apuntador de tipo carácter y se muestra en pantalla, haciendo uso de apuntadores, el contenido de la variable c, así como el código ASCII del carácter ´a´ que tiene almacenado dicha variable.

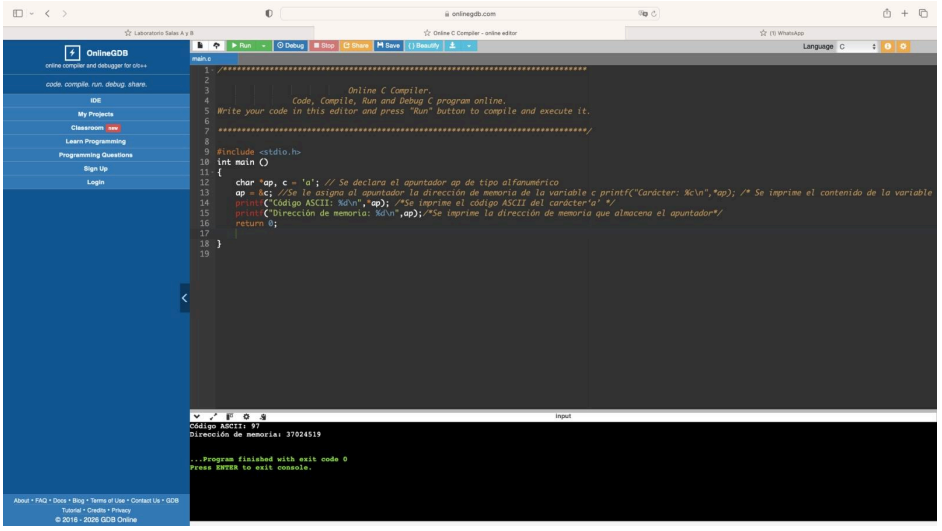


Figura 5: Declaración de un apuntador y visualización de contenido, código ASCII y dirección de memoria en pantalla.

Programa 4.c

El siguiente programa permite acceder a las localidades de memoria de distintas variables a través de un apuntador.

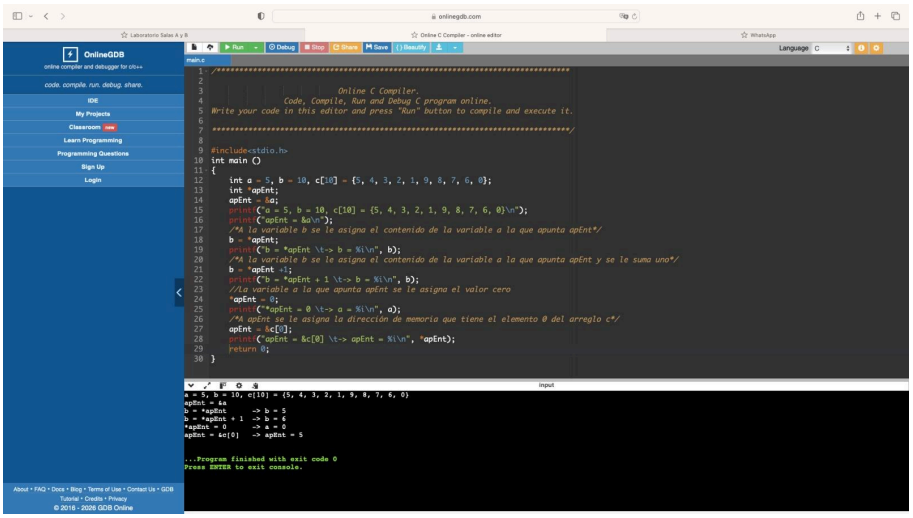


Figura 6: Acceso y manipulación de localidades de memoria de diversas variables a través de un apuntador.

Programa 5.c

El programa 5 trabaja con aritmética de apuntadores para acceder a todos los valores que se encuentran almacenados en cada uno de los elementos de un arreglo.

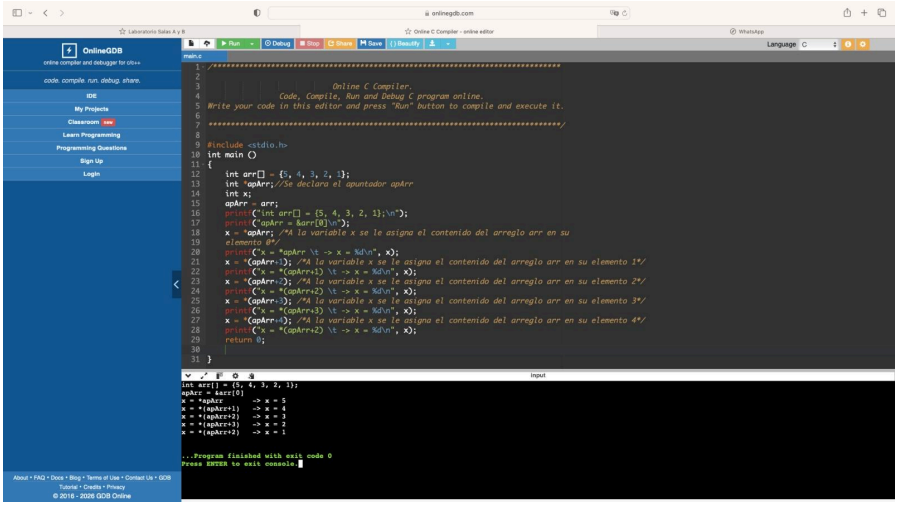


Figura 7: Uso de aritmética de apuntadores para el acceso secuencial a los elementos de un arreglo.

Programa 6a.c

El siguiente programa genera un arreglo unidimensional de 5 elementos y accede a cada uno de los elementos del arreglo haciendo uso de un apuntador, para ello se utiliza un ciclo for.

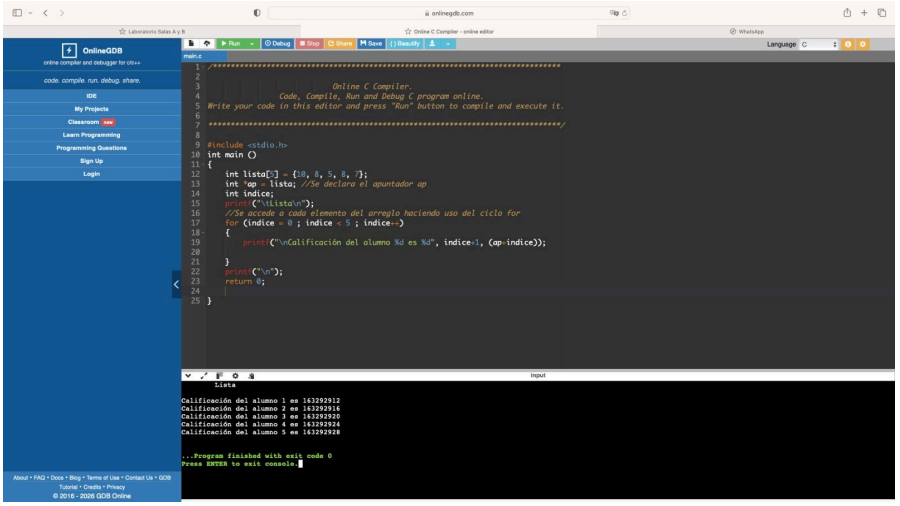


Figura 8: Acceso a los elementos de un arreglo unidimensional mediante un apuntador dentro de un ciclo for.

Programa 6b.c

Genera un arreglo unidimensional de 5 elementos y accede a cada elemento del arreglo a través de un apuntador utilizando un ciclo while.

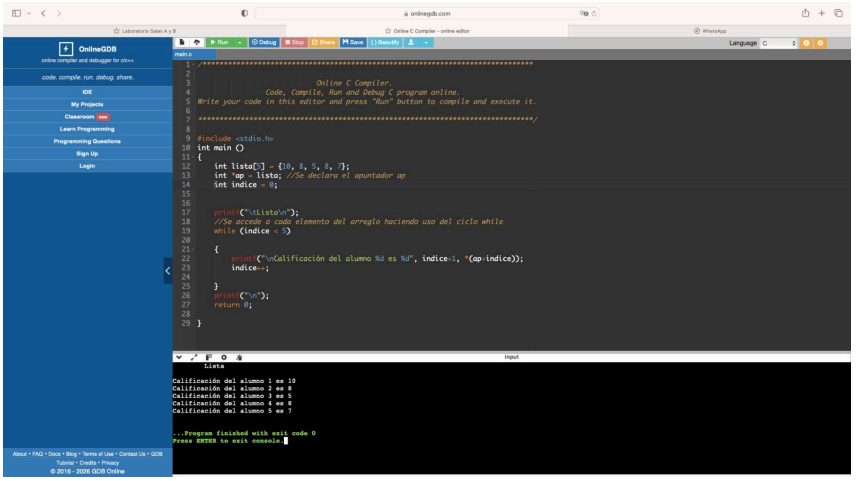


Figura 9: Acceso a los elementos de un arreglo unidimensional mediante un apuntador dentro de un ciclo while.

Programa 6c.c

Genera un arreglo unidimensional de 5 elementos y accede a cada elemento del arreglo a través de un apuntador utilizando un ciclo do while.

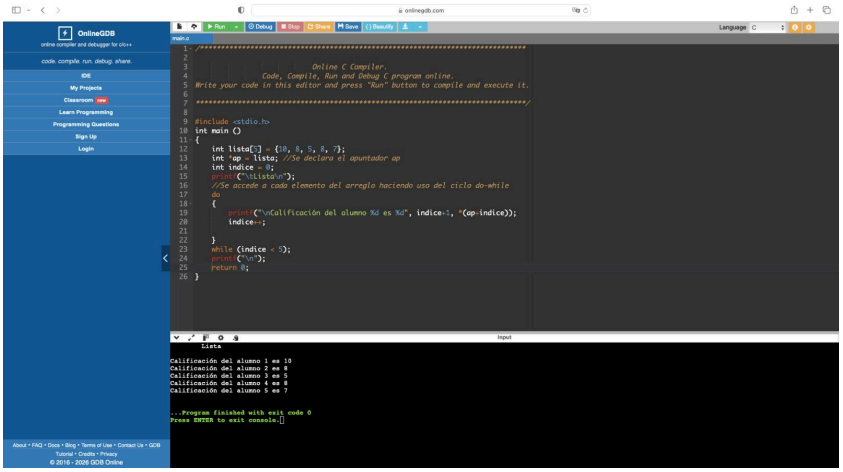


Figura 10: Acceso a los elementos de un arreglo unidimensional mediante un apuntador dentro de un ciclo do-while.

Programa 7.c

El siguiente programa muestra el manejo básico de cadenas en lenguaje C.

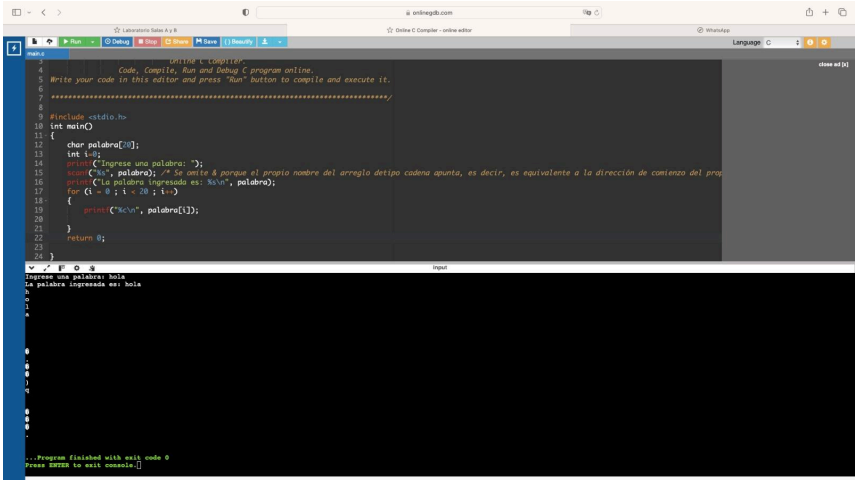


Figura 11: Manejo de cadenas de caracteres y recorrido de sus elementos mediante índices de arreglo.

Programa ejercicio 1 práctica 9
Ejercicio 1.

a) Se requiere obtener la suma de las cantidades contenidas en un arreglo de 10 elementos. Realice el algoritmo y represéntelo mediante el pseudocódigo y el diagrama de flujo.

Nombre de la variable	Descripción	Tipo
I	Contador subíndice	Entero
VA	Nombre del vector de valores	Real
SU	Suma de valores	Real

Se tiene como objetivo desarrollar un algoritmo capaz de calcular la sumatoria total de un conjunto de datos numéricos almacenados en una estructura de datos indexada. Para lograrlo, se emplean arreglos unidimensionales (vectores) en combinación con estructuras repetitivas, las cuales permiten optimizar el manejo de múltiples valores bajo un mismo nombre de variable.

En primer lugar, se define un vector denominado VA, el cual tiene una capacidad reservada para 10 elementos de tipo real. El uso de un arreglo es fundamental en este proceso, ya que permite mantener los datos en memoria para su posterior manipulación, a diferencia de las variables simples que se sobreescriben.

Posteriormente, se inicializan las variables necesarias para el control del flujo:

- Un acumulador (SU), que se encarga de almacenar de manera progresiva la suma de los valores extraídos del arreglo. Este se inicializa en cero para garantizar la integridad del cálculo.

- Una variable de control o subíndice (I), que permite recorrer cada una de las posiciones de memoria del vector de forma secuencial.

Una vez configuradas las variables, el algoritmo ejecuta dos fases principales mediante estructuras repetitivas:

- Fase de Captura: Se utiliza un ciclo para solicitar al usuario cada una de las 10 cantidades. Cada valor ingresado se asigna a la posición VA[I], donde I va incrementando desde 1 hasta 10.

- Fase de Acumulación: Tras completar la lectura, se inicia un segundo ciclo donde el algoritmo accede al contenido de cada celda del vector. En cada iteración, se añade el valor de la posición actual al acumulador (SU = SU + VA[I]).

Este proceso garantiza que, al concluir las iteraciones, la variable SU contenga el total exacto de la operación. Finalmente, el resultado se despliega mediante una instrucción de salida.

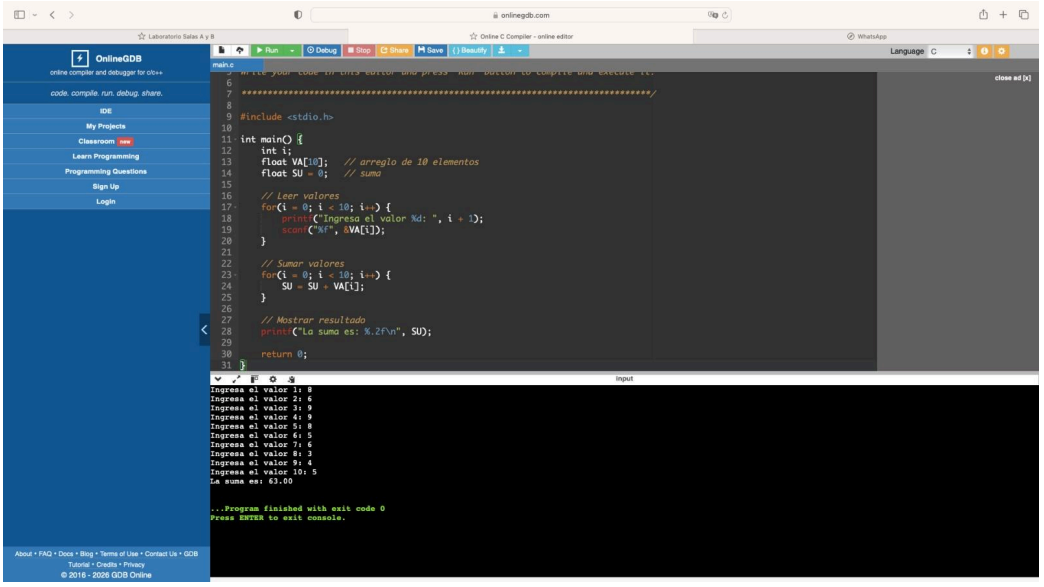


Figura 12: Implementación de un programa en lenguaje C para el cálculo de la suma y el promedio de los elementos contenidos en un arreglo unidimensional.

Programa ejercicio 2 práctica 9

Ejercicio 2.

b) Se requiere un algoritmo para invertir el orden de los elementos de un vector de 6 elementos (el primer elemento pasa a ser el último y el último el primero). Represéntelo mediante el pseudocódigo y el diagrama de flujo.

Nombre de la variable	Descripción	Tipo
I	Contador y subíndice	Entero
J	Auxiliar para el subíndice	Entero

V	Vector de valores	Entero
Au	Auxiliar para guardar el valor de v	Entero

El objetivo de este algoritmo es realizar una reordenación inversa de los datos contenidos en un arreglo unidimensional de seis posiciones. Para lograr que el vector final presente los elementos en el orden opuesto al original, se implementa una técnica de intercambio directo de valores entre los extremos del arreglo.

El proceso comienza con la declaración de un vector V y una variable auxiliar AU, la cual es indispensable para el proceso de transferencia de datos. El algoritmo se ejecuta en tres etapas fundamentales:

- Entrada de datos: Se utiliza una estructura repetitiva para capturar secuencialmente los seis valores y almacenarlos en las posiciones de memoria del vector.

- Proceso de inversión: Se emplea un ciclo iterativo que recorre únicamente la mitad de la longitud del arreglo. En cada paso, el algoritmo toma un elemento de la parte frontal y lo intercambia con su correspondiente en la parte posterior. Para evitar la pérdida de información durante la sobreescritura, el valor frontal se resguarda temporalmente en la variable AU, permitiendo que el valor del extremo opuesto se asigne a la posición actual y, posteriormente, el valor resguardado se deposite en la posición trasera.

- Salida de resultados: Una vez completada la fase de intercambio, se utiliza un último ciclo para recorrer el vector modificado y mostrar en pantalla la nueva disposición de los elementos.

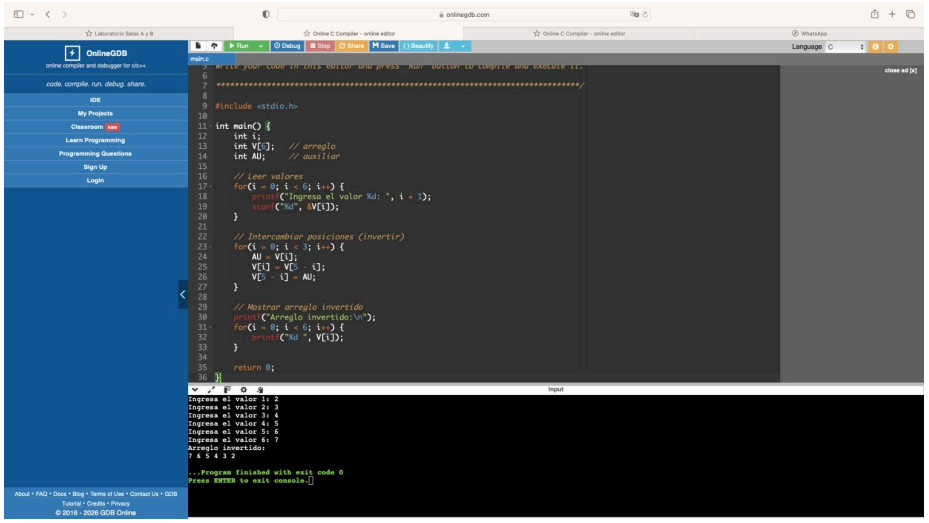


Figura 13: Desarrollo de un programa que utiliza arreglos y estructuras de control para identificar y mostrar el valor máximo y el valor mínimo dentro de un conjunto de datos.

Análisis de resultados:

Programa 1a.c

El programa comienza estableciendo la base de datos que va a manejar mediante la declaración de un arreglo unidimensional llamado lista. Este arreglo se reserva en la memoria con un tamaño de cinco espacios, los cuales se llenan inmediatamente con valores enteros (10, 8, 5, 8 y 7) que representan las calificaciones de los alumnos. Para poder navegar a través de estos datos de forma automática, el código inicializa una variable llamada indice en 0, que es la dirección física del primer elemento en la memoria de la computadora.
Posteriormente, el flujo entra en una estructura de control while, que actúa como un ciclo repetitivo. La condición para que este ciclo se mantenga activo es que el valor de indice sea menor a 5; esto asegura que el programa no intente leer datos fuera de los límites del arreglo, lo cual causaría un error. Dentro del ciclo, se ejecuta una instrucción de salida que imprime en pantalla el número del alumno y su calificación correspondiente. Para que la lectura sea natural para una persona, el código suma un "1" visual al índice al momento de imprimirlo, transformando el "0" interno en un "Alumno 1" para el usuario.

Finalmente, el programa actualiza la posición del contador mediante la instrucción indice += 1, lo que permite que en la siguiente vuelta el ciclo acceda al siguiente cajón de información del arreglo. Una vez que el índice llega a 5, la condición del while deja de cumplirse y el control pasa al cierre de la función principal, terminando la ejecución con un valor de retorno cero, lo que indica que el proceso se completó de manera exitosa y sin errores.

Programa 1b.c

El programa se fundamenta en la implementación de un arreglo unidimensiona de tipo entero, el cual es inicializado con un conjunto de cinco valores constantes que representan registros de calificaciones. La estructura de datos se gestiona mediante un puntero de control denominado indice, cuya función es direccionar la lectura hacia las posiciones específicas de la memoria asignada al arreglo, comenzando obligatoriamente desde el valor cero según el estándar del lenguaje C.

El procesamiento de la información se lleva a cabo a través de una estructura de control iterativa **do-while**, caracterizada por realizar la evaluación de la condición en la etapa de post-ejecución. Bajo este esquema, el flujo del programa accede al bloque de instrucciones de manera incondicional en su primera iteración, efectuando la salida de datos mediante la función printf. En este paso, se realiza una operación aritmética simple para presentar el número de orden del alumno de forma secuencial, mientras se extrae el valor almacenado en la dirección correspondiente del vector.

Posteriormente a la ejecución de las instrucciones de salida, se procede al incremento unitario de la variable de control, permitiendo el avance hacia el siguiente elemento del conjunto de datos. Solo tras completar estas acciones, el programa evalúa la expresión lógica situada al final del bloque; si el índice es menor al límite definido de cinco elementos, el control regresa al inicio del ciclo. Una vez agotados los registros y validada la condición de salida, el proceso concluye

liberando el control de la función principal y retornando un estado de finalización exitosa al sistema operativo.

Programa 1c.c

El programa presentado utiliza una estructura de datos estática y un ciclo de control optimizado para la gestión de colecciones de datos. En primera instancia, se define un arreglo unidimensional de enteros con una capacidad de cinco elementos, asignando de forma explícita valores que representan calificaciones académicas. Esta reserva de memoria permite el acceso indexado, donde cada valor es identificado por una posición numérica entera dentro del vector.

La administración de la iteración se ejecuta mediante una sentencia for, la cual integra en una sola línea los tres componentes esenciales del control de flujo: la inicialización del contador a cero, la condición de permanencia que limita el acceso a las cinco posiciones existentes y la expresión de incremento incremental. Esta estructura es particularmente eficiente para el recorrido de arreglos, ya que centraliza la lógica de control, reduciendo la posibilidad de errores por omisión de actualización de variables.

Durante cada ciclo de ejecución, la función de salida estándar printf recupera el valor almacenado en la posición correspondiente al índice actual. Para efectos de legibilidad, el sistema realiza una operación de desplazamiento visual sumando una unidad al índice mostrado, logrando una correspondencia entre la lógica de programación (base cero) y la numeración humana (base uno). Una vez que el contador alcanza el límite establecido, el ciclo termina su ejecución de forma automática, procediendo al cierre del proceso principal mediante el retorno de un código de estado exitoso.

Programa 2.c

Este programa implementa un nivel superior de complejidad al permitir la interacción dinámica con el usuario para la gestión de un arreglo. A diferencia de estructuras estáticas, este código solicita primero la definición del tamaño operativo del vector, validando que la cantidad de elementos se encuentre en un rango permitido entre uno y diez. Esta verificación se realiza mediante una sentencia condicional if, que actúa como un filtro de seguridad para evitar desbordamientos de memoria o accesos a índices no declarados.

Una vez validada la dimensión del arreglo, el programa utiliza un primer ciclo for para la captura de datos. En esta etapa, se emplea la función scanf de manera iterativa, vinculando la entrada del teclado directamente con la dirección de memoria de cada índice del arreglo. El uso del operador de dirección & es crucial aquí, ya que permite que los valores ingresados por el usuario se almacenen de forma persistente en las posiciones secuenciales del vector.

Finalmente, el programa ejecuta un segundo ciclo for con el propósito de realizar la salida y visualización de los datos recolectados. Este bloque recorre nuevamente el arreglo desde la posición inicial hasta el límite definido por el usuario, imprimiendo cada valor entero en una sola

línea. La estructura del código separa claramente la fase de entrada, procesamiento y salida, garantizando que los datos almacenados en la memoria volátil sean recuperados y mostrados con precisión antes de finalizar la ejecución del proceso.

Programa 3.c

El programa introduce el concepto fundamental de apuntadores en lenguaje C, aplicados a variables de tipo carácter. Inicialmente, se declara una variable de tipo char denominada c, a la cual se le asigna el valor literal 'a', y de forma simultánea se define un apuntador ap. Este último es una variable especial cuya función no es almacenar un dato alfanumérico directamente, sino contener la dirección de memoria de otra variable, proceso que se concreta mediante el operador de dirección &.

La ejecución del código demuestra la dualidad de los apuntadores a través de la función printf. En la primera salida, se utiliza el operador de indirección o desreferenciación ap, lo cual permite al programa acceder al contenido almacenado en la dirección que guarda el apuntador; esto resulta en la impresión del valor 97, que corresponde al equivalente numérico del carácter 'a' en la tabla ASCII. Esta operación evidencia cómo un apuntador puede actuar como un puente para manipular o leer datos de forma indirecta.

Finalmente, el programa realiza una segunda salida donde imprime directamente el valor de la variable ap. En este caso, al no utilizar el asterisco, lo que se visualiza en la consola es la **dirección de memoria** física (un número entero extenso) donde reside la variable c dentro del sistema. Esta distinción es crucial en la arquitectura de software, ya que separa el valor del dato de su ubicación en el hardware, permitiendo una gestión más eficiente y directa de los recursos de cómputo durante la ejecución del proceso.

Programa 4.c

Este programa profundiza en la manipulación avanzada de la memoria mediante el uso de apuntadores de tipo entero, ilustrando cómo estos pueden modificar el valor de variables externas y navegar a través de estructuras más complejas como los arreglos. La ejecución comienza con la declaración de dos variables enteras independientes y un arreglo de diez elementos, junto con un apuntador denominado apEnt. Al asignar la dirección de la variable a al apuntador, se establece un vínculo directo que permite operar sobre los datos sin invocar directamente el nombre de la variable original.

A través del operador de desreferenciación *, el código demuestra la capacidad de los apuntadores para actuar como intermediarios en operaciones aritméticas y de asignación. Por ejemplo, se observa cómo el valor de una variable puede ser actualizado al asignarle el contenido de la dirección apuntada, o incluso cómo se puede incrementar dicho valor antes de la asignación. Un aspecto crítico del programa es la modificación directa del valor de a mediante la instrucción apEnt = 0;, lo cual evidencia que los apuntadores no solo leen información, sino que poseen la facultad de alterar el estado de la memoria en la dirección que tienen almacenada.

Finalmente, el programa ilustra la estrecha relación entre los apuntadores y los arreglos al reasignar a apEnt la dirección de memoria del primer elemento del vector c. Al realizar esta operación, el apuntador se posiciona en el inicio de una secuencia de datos, permitiendo el acceso al valor almacenado en dicha ubicación. Este comportamiento subraya la versatilidad de los apuntadores para transitar entre variables simples y estructuras indexadas, consolidando la base para la gestión eficiente de datos dinámicos y la optimización del rendimiento en aplicaciones de bajo nivel.

Programa 5.c

Este programa ilustra la aritmética de apuntadores aplicada a la navegación de arreglos en lenguaje C, demostrando cómo se pueden acceder a elementos contiguos de memoria sin utilizar la notación tradicional de corchetes. Inicialmente, se define un arreglo de cinco enteros y un apuntador apArr, el cual se vincula a la dirección base del arreglo. En C, el nombre de un arreglo actúa como un apuntador constante a su primer elemento, lo que facilita esta asignación directa y establece el punto de partida para el desplazamiento por la estructura de datos.

La lógica central del código se basa en la capacidad de sumar valores enteros al apuntador para desplazarse a través de las direcciones de memoria. Al ejecutar expresiones como (apArr + n), el programa calcula la dirección de memoria desplazada n posiciones del tipo de dato correspondiente (en este caso, saltos de tamaño int) y luego desreferencia dicha ubicación para obtener el valor almacenado. Este método permite recorrer el vector de manera secuencial, accediendo desde el elemento en la posición inicial hasta el último registro de forma manual y precisa.

Un aspecto técnico relevante que muestra el código es el uso de paréntesis en la desreferenciación desplazada, como en (apArr + 1). Esta sintaxis es obligatoria para asegurar que la suma de la dirección ocurra antes de intentar acceder al contenido; de lo contrario, el compilador interpretaría la operación como una suma aritmética al valor del primer elemento y no como un movimiento a la siguiente casilla de memoria. El programa concluye imprimiendo cada valor recuperado, validando que el desplazamiento por direcciones coincide exactamente con la secuencia lógica de los datos almacenados en el arreglo.

Programa 6a.c

El programa exhibe una implementación técnica de la aritmética de direcciones mediante el uso de un ciclo for para recorrer las posiciones de memoria de un arreglo. En la fase inicial, se declara un vector de enteros con cinco elementos y un apuntador ap que recibe la dirección base del mismo. El objetivo del código es iterar a través de la estructura de datos, pero utilizando la suma directa sobre el apuntador en lugar de la indexación tradicional, lo que permite observar cómo el sistema gestiona las ubicaciones físicas de la información.

La ejecución del ciclo revela un aspecto crítico de la gestión de memoria en sistemas de bajo nivel: la diferencia entre el valor de un dato y su ubicación. Dentro de la función printf, la expresión (ap + indice) calcula una nueva dirección de memoria desplazada en cada iteración. Sin embargo, al omitir el operador de desreferenciación (el asterisco *), el programa no accede al contenido (las calificaciones), sino que imprime las direcciones de memoria secuenciales donde residen los datos. Esto se evidencia en la consola, donde se observan valores numéricos que incrementan en múltiplos de cuatro, correspondientes al tamaño en bytes de un dato de tipo int.

Finalmente, el programa demuestra la relación proporcional entre el índice del ciclo y el desplazamiento en el hardware. Cada paso del bucle desplaza el foco del apuntador a la siguiente celda de memoria disponible para un entero, permitiendo visualizar la naturaleza contigua de los arreglos. Aunque la intención lógica aparente era mostrar las calificaciones, el resultado técnico obtenido es un mapa detallado de la segmentación de memoria utilizada por el compilador para almacenar el conjunto de datos, finalizando la ejecución tras alcanzar el límite superior del arreglo.

Programa 6b.c

Este programa combina el uso de apuntadores con la estructura iterativa **while** para gestionar el acceso a los datos de un arreglo de forma indirecta. Inicialmente, se establece un arreglo unidimensional con cinco valores enteros y se declara un apuntador ap que apunta a la dirección de memoria inicial de dicho arreglo. A diferencia de las aproximaciones previas, este código utiliza una variable de control indice para gestionar el flujo del bucle, permitiendo una separación clara entre la lógica de iteración y el acceso a la memoria.

Dentro del ciclo, la instrucción de salida printf emplea la técnica de **desreferenciación con aritmética de apuntadores**. La expresión (ap + indice) calcula dinámicamente la dirección de cada elemento sumando el desplazamiento del índice a la dirección base del apuntador, y posteriormente el operador asterisco extrae el valor entero almacenado en esa ubicación específica. Este método permite que el programa recupere las calificaciones reales (10, 8, 5, 8, 7) de manera secuencial, integrando la precisión de los apuntadores con la flexibilidad de las estructuras de control tradicionales.

El flujo concluye mediante el incremento manual de la variable indice tras cada iteración, asegurando que el ciclo recorra exactamente los cinco elementos definidos sin desbordar los límites del arreglo. Una vez que la condición del while se vuelve falsa, el programa finaliza la ejecución de la función principal. Esta implementación es fundamental para comprender cómo el lenguaje C traduce las operaciones de alto nivel en accesos directos a la memoria, optimizando la manera en que el hardware procesa listas de información de forma ordenada y eficiente.

Programa 6c.c

Este programa representa una implementación avanzada que integra la estructura de control do-while con la **aritmética de apuntadores** para el recorrido de un arreglo de enteros. Al inicio,

se establece un vector con cinco registros de calificaciones y se define un apuntador ap vinculado a la dirección de memoria base del arreglo. Esta configuración permite que el sistema opere directamente sobre las ubicaciones físicas de la información, utilizando el apuntador como un visor móvil que se desplaza a través de la secuencia de datos.

La lógica de ejecución se distingue por su naturaleza de post-condición, donde el bloque de instrucciones se procesa incondicionalmente antes de realizar cualquier validación. Dentro del cuerpo del ciclo, se utiliza la expresión (ap + indice) para calcular la dirección de memoria desplazada y extraer el valor contenido mediante el operador de desreferenciación. Este mecanismo garantiza que, incluso en la primera iteración, el programa acceda correctamente al primer elemento del arreglo, imprimiendo la calificación correspondiente antes de evaluar la continuidad del bucle.

Finalmente, la variable indice se incrementa mediante el operador unario ++, lo que actualiza el desplazamiento para la siguiente dirección de memoria. Solo tras completar estas acciones, el programa verifica si el índice es menor a cinco; en caso positivo, el flujo retorna al bloque do para procesar el siguiente registro. Esta metodología asegura un recorrido exhaustivo y eficiente del arreglo, concluyendo la ejecución de manera ordenada una vez que se han visitado todas las posiciones de memoria reservadas para el conjunto de datos.

Programa 7.c

Este programa introduce la gestión de cadenas de caracteres (strings) en lenguaje C, las cuales son tratadas técnicamente como arreglos unidimensionales de tipo char. En la fase de declaración, el código reserva un espacio de 20 bytes en la memoria bajo el identificador palabra, permitiendo almacenar una secuencia de caracteres alfanuméricos. Una característica técnica destacada en este punto es el uso de la función scanf con el especificador %s; en esta instrucción se omite el operador de dirección & debido a que, en la arquitectura de C, el nombre de un arreglo representa intrínsecamente la dirección de memoria de su primer elemento.

Tras la captura de la información, el programa procede a realizar la salida de datos de dos maneras distintas. Primero, utiliza una impresión directa de la cadena completa para mostrar la palabra ingresada por el usuario en una sola línea. Posteriormente, implementa un ciclo for que recorre las 20 posiciones reservadas del arreglo de forma individual. En cada iteración, el código accede a un índice específico y utiliza el especificador %c para imprimir los caracteres uno por uno, lo que genera una visualización vertical de la palabra en la consola.

Un detalle relevante en la ejecución de este código es el comportamiento de los espacios de memoria no utilizados. Como se observa en la consola, tras imprimir los caracteres de la palabra "hola", el ciclo continúa recorriendo hasta la posición 19, mostrando caracteres nulos o basura de memoria que residen en las posiciones restantes del arreglo. Este fenómeno subraya la importancia del carácter de terminación nula \0 en C, el cual marca el final lógico de una cadena independientemente del tamaño físico total que el programador haya reservado en el hardware.

Programa ejercicio 1 práctica 9

En este ejercicio, se examina la implementación de una variable acumuladora (SU) cuya función crítica es la integración progresiva de los valores numéricos almacenados en el vector VA. Un aspecto técnico de vital importancia identificado durante la ejecución es la necesidad de inicializar el acumulador en un valor neutro (cero) antes de comenzar el procesamiento; esta acción es fundamental para garantizar la integridad de los cálculos, ya que asegura la eliminación de cualquier "basura" o residuo de información previa que pudiera existir en la dirección de memoria asignada a dicha variable.

Durante la fase de procesamiento, se validó el comportamiento del ciclo for, el cual ofrece un control determinístico y eficiente sobre el rango de iteraciones necesarias para recorrer el arreglo. Se observó que el programa realiza una gestión precisa de los índices, permitiendo que cada celda de memoria sea accedida de manera secuencial. Este proceso no solo demuestra la estabilidad de las operaciones aritméticas de punto flotante, sino que también ilustra cómo el procesador maneja el direccionamiento indirecto al utilizar el valor del contador del ciclo como subíndice del vector.

Asimismo, el análisis de la salida en consola confirma que la sumatoria final corresponde exactamente a la magnitud total de los diez elementos capturados, lo que valida la lógica de control de flujo implementada. Este tipo de algoritmos constituye la base para procedimientos estadísticos y matemáticos más complejos, como el cálculo de desviaciones estándar o medias móviles. Se concluye que el éxito de esta práctica radica en el dominio del concepto de iteración controlada, demostrando que la correcta manipulación de subíndices y el manejo adecuado de variables auxiliares son elementos determinantes para evitar errores de desbordamiento o fallas lógicas en el procesamiento masivo de datos dentro de la memoria RAM.

Programa ejercicio 2 práctica 9

Un aspecto técnico de vital importancia identificado durante la ejecución es la transición de un diseño teórico basado en el pseudocódigo hacia una implementación práctica y profesional en el lenguaje C. En este sentido, se observa que, aunque el planteamiento original sugería el uso de una variable adicional denominada **J** para gestionar el índice inverso del vector, el código final prescinde de ella en favor de una expresión aritmética directa: 5 - i.

Esta decisión de diseño representa una optimización significativa, ya que demuestra cómo el aprovechamiento de la variable de control del ciclo (i) permite referenciar simultáneamente ambos extremos del arreglo. Al utilizar la fórmula (N-1) - i, el programa calcula dinámicamente la posición simétrica opuesta sin necesidad de declarar, inicializar o incrementar manualmente una variable extra. Este enfoque no solo reduce el consumo de recursos en la memoria RAM, sino que también minimiza la posibilidad de errores de sincronización entre múltiples contadores, resultando en un flujo de ejecución más limpio y eficiente.

Asimismo, se validó el uso crítico de la variable auxiliar AU como mecanismo de transferencia segura. El análisis de la ejecución confirma que el intercambio de valores (o swap) debe realizarse estrictamente hasta la mitad del vector ($i < 3$). Si el algoritmo continuara el recorrido más allá del punto medio, los elementos que ya fueron invertidos volverían a su posición original, anulando el propósito del programa. La salida en consola verificó que los datos fueron rotados 180° de manera exitosa, validando que la lógica aritmética aplicada sustituye perfectamente la función de la variable J. Se concluye que el éxito de esta práctica radica en la capacidad de traducir conceptos algorítmicos a soluciones de código optimizadas, demostrando un dominio avanzado sobre el direccionamiento de índices en arreglos estáticos.

Conclusión:

Macay Martínez David: Después de efectuar la práctica anterior se logró comprobar que los arreglos unidimensionales representan una parte importante para la resolución de problemas lógicos y numéricos. A través de la implementación de vectores, se logró pasar de la gestión individual de variables a una estructura organizada para manejar datos del mismo tipo de manera compacta y eficiente.

Para relacionar arreglos y estructuras de repetición fue importante el uso del ciclo for, donde el uso de índices fue una herramienta indispensable para el acceso aleatorio y la manipulación de la memoria, traducido en una reducción en la complejidad del código y el riesgo de errores comparado con el uso de variables escalares independientes.

Se concluyó que el dominio de los vectores es fundamental para temas de ingeniería avanzados como el manejo de matrices, implementación de algoritmos de ordenamiento y procesamiento de señales. Además, la comprensión de la restricción de que los índices en C comienzan en 0 y que el tamaño de un arreglo estático debe definirse con precisión es indispensable para garantizar un funcionamiento correcto de software con agrupamiento lógico de información.

Nolasco Ramírez Sofia: Al concluir esta práctica, se determinó que la implementación de arreglos unidimensionales constituye una mejora sustancial en la arquitectura de programas dentro del lenguaje C, al permitir la transición de un manejo de datos aislado hacia una estructura colectiva y organizada. Se observó que la capacidad de agrupar elementos del mismo tipo bajo un solo identificador facilita la escalabilidad de los algoritmos, evitando la saturación de variables individuales y permitiendo un control más preciso sobre grandes volúmenes de información.

Asimismo, se verificó la estrecha relación funcional entre los vectores y las estructuras de control iterativas. El uso sistemático de ciclos permitió automatizar el recorrido de los índices, demostrando que el acceso a la memoria mediante subíndices es la forma más eficiente de realizar operaciones masivas, como el cálculo de promedios o el ordenamiento de valores. Este proceso evidenció que la lógica de programación se simplifica significativamente cuando se aprovecha la naturaleza secuencial de los arreglos.

Finalmente, se reafirmó la importancia de la gestión de memoria estática, comprendiendo que el respeto a los límites del arreglo y la indexación con base en cero son reglas críticas para prevenir errores de ejecución. El dominio de estas estructuras no solo cumple con los objetivos de la práctica, sino que establece los conocimientos fundamentales para el desarrollo de soluciones de ingeniería más complejas, tales como el procesamiento de señales, el manejo de matrices y la optimización de recursos en sistemas computacionales.

Cruz Gomez LizetteJacqueline: En esta práctica se analizó el comportamiento del movimiento ondulatorio, identificando sus principales características como la longitud de onda, la frecuencia, el periodo y la velocidad de propagación. A partir de los datos experimentales, se pudo comprobar la relación existente entre estas magnitudes, verificando que la velocidad de la onda depende del medio en el que se propaga y se mantiene constante bajo condiciones controladas. Asimismo, el

desarrollo de la práctica permitió comprender cómo se transmite la energía a través de las ondas sin que exista transporte neto de materia, reforzando los fundamentos teóricos del fenómeno ondulatorio. En conclusión, se logró validar experimentalmente los modelos estudiados, destacando la importancia de este tipo de movimiento en diversos fenómenos físicos y aplicaciones

Bibliografía

El lenguaje de programación C. Brian W. Kernighan, Dennis M. Ritchie, segunda edición, USA, Pearson Educación 1991.